Paper 32 - The Supervisor Cursor

CQE-paper-32
CQECMPLX Formal Paper Series
Review-facing PDF built 2026-06-13

Construction Basis

Use superpermutation schedules as supervisor cursors: compressed action graphs that enumerate every requested ordering while producing deployable PermForge/GraphStax machinery.

This PDF is the clean paper artifact. Source extracts, kernels, receipts, tool notes, workbook material, and package bindings are used as construction evidence; they are not treated as separate review-facing papers.

Evidence Inputs

- promoted formal paper: production\formal-papers\CQE-paper-32\FORMAL_PAPER.md
- paper kernel manifest: production\paper-kernels\papers\CQE-paper-32\paper_kernel_manifest.json
- verifier: production\formal-papers\CQE-paper-32\verify_supervisor_cursor_schedule.py
- receipt: production\formal-papers\CQE-paper-32\supervisor_cursor_schedule_receipt.json
(CQE-paper-32: pass_with_open_minimality_obligations)
- body: production\papers\CQE-paper-32\PAPER-BODY.md
- source: production\papers\CQE-paper-32\SOURCE.md
- formal: production\papers\CQE-paper-32\01-CQE-formal\FORMAL.md
- tool: production\papers\CQE-paper-32\02-CQE-tool\TOOL.md
- workbook: production\papers\CQE-paper-32\03-CQE-workbook\WORKBOOK.md
- recovered_output: production\lib-forge\recovered\papers_output\CQE-paper-32.md

Paper 32 - The Supervisor Cursor

Abstract

Paper 32 packages superpermutation schedules as supervisor cursors. A superpermutation is a string whose length-`n` windows cover every ordering of `n` symbols. In this corpus, that string is not proof content by itself. It is a compressed request schedule: a cursor that decides which enumeration request fires next.

The closed result is practical and bounded. The verifier validates superpermutation coverage records for `n=4` through `n=8`, confirms the recursive chart-walk construction at `n=8`, records the Egan upper row `46205`, records the lower-bound corridor of `120`, checks the n=5 octad structure, and confirms that the paper-kernel selector wraps Paper 32 forward to Paper 01 for active-suite retest. It claims minimality only where the local evidence permits it: `n=4` and `n=5`. It does not claim minimality for `n>=6`.

Closed Evidence

The receipt is generated by `production/formal-papers/CQE-paper-32/verify_supervisor_cursor_schedule.py`. It imports the live `GraphStax.permforge` package surface and writes `supervisor_cursor_schedule_receipt.json`.

The verifier calls `verify_record(n)` for `n=4..8`. Every record has validated coverage:

```
n=4: length 33, coverage true, minimality closed
n=5: length 153, coverage true, minimality closed
n=6: length 873, coverage true, minimality not claimed
n=7: length 5908, coverage true, minimality not claimed
n=8: length 46205, coverage true, minimality not claimed
```

The verifier also constructs the recursive chart-walk string for `n=8`. Its length is `46233`, and coverage is true. The shipped `n=8` record matches the Egan upper formula at `46205`, while the lower bound is `46085`. The open corridor is therefore `120`.

Finally, the verifier checks the scheduler surface. A `SuperPermScheduler` dispatches enumeration requests over items. The cursor is not ribbon content; it is the request schedule that tells the kernel which local ordering to read.

Definitions

An enumeration request is a cursor event. Without a request, the center `C` is not produced for the readout.

A local chart is a length-`n` window of the schedule string.

Coverage means that the set of length-`n` windows contains every permutation of the `n` symbols.

Minimality means no shorter covering string exists. This paper treats minimality as closed only for `n=4` and `n=5`.

A supervisor cursor is a compressed schedule for requests. It tells the suite which ordering to inspect next; it does not replace the receipt attached to the thing being inspected.

A selector is a deployable paper-kernel route: suite, block, or individual paper.

Claims

- Coverage for the shipped `n=4..8` schedule records is validated.
- The recursive chart-walk construction reaches `n=8` with length `46233` and full coverage.
- The shipped `n=8` record has length `46205`, matching the Egan upper row.
- The `n=8` corridor between the lower bound and the Egan upper row has width `120`.
- The n=5 octad has eight schedules and a reversal orbit with four fixed points and two swapped pairs.
- The supervisor cursor schedules enumeration requests. It is not ribbon content and not hidden proof support.
- The paper-kernel selector wraps Paper 32 forward to Paper 01 for active suite retest, while Paper 00 remains the inherited method contract.

Theorem 32

The suite can be packaged with a supervisor cursor when the cursor is treated as a compressed request schedule over validated coverage rows, while each paper's proof/open/readout status remains attached to its own receipt.

Proof

Run `verify_supervisor_cursor_schedule.py`.

The coverage checks pass because `verify_record(n)` returns validated records with coverage true for every `n` from 4 through 8.

The minimality-scope check passes because the verifier marks only `n=4` and `n=5` as closed minimality rows. For `n=6`, `n=7`, and `n=8`, the records are validated schedules and bounds rows, not minimality proofs.

The `n=8` bounds checks pass because the shipped record length equals the Egan upper value `46205`; the lower bound is `46085`; and the open corridor is `120`. The recursive chart-walk construction also covers at length `46233`.

The scheduler check passes because `SuperPermScheduler(4)` dispatches item requests from the superpermutation string and reports that the cursor is not content. This closes the "No request, no C" packaging rule.

The kernel-selector check passes because the paper-kernel registry places Paper 32 at the suite wrap: the next active-suite paper is Paper 01. Therefore Paper 32 can serve as a deployable supervisor cursor without hiding proof status. This proves Theorem 32.

Worked Example

Take the eight Paper 30 ribbon slots:

```
C, L, R, B, T, O, W, A
```

The naive way to inspect all orderings of these eight slots is to write every ordering independently. The schedule way is to walk a superpermutation string: each length-8 window is

one local read order, and overlapping windows reuse symbols.

The verifier checks the shipped `n=8` schedule at length `46205`. Every one of the `8! = 40320` orderings appears. The result is not a proof that `46205` is minimal; it is a validated coverage record at the Egan upper row, with an open minimality corridor of `120`.

When the cursor fires, it asks for an ordering. The target paper or tool still brings its own receipt. The cursor only decides the request order.

Open Obligations

Minimality for `n>=6` remains open unless independent shorter-string exclusion proofs are supplied.

The `n=8` corridor below `46205` remains open. Any shorter candidate must ship with a coverage receipt and falsifier rows.

Product selectors must preserve proof/open/readout status. A cursor that makes navigation easier may not hide obligations.

Older "final observation" language remains reflective only unless a separate formal claim and verifier are supplied.

Suite Role

Paper 32 is the deployable selector layer for the paper suite. It lets the kernel route suite, block, and paper requests while preserving status. Its wrap to Paper 01 makes the active suite retestable as a loop; Paper 00 remains the inherited contract outside that active proof window.